Open Access | https://doi.org/10.25080/yxan2279



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Everything That Breaks When You Put an LLM Agent in Production

Sruthi Pisipati[1], and **Haris Javed**[1]

[1]Boston Scientific Corporation

Abstract

Agentic AI systems often perform well in interactive prototypes and small evaluations, but they behave like distributed systems when deployed in production-scale workflows. We present a case study from a regulated record-processing environment where LLM-based agents were used for structured classification in two related workflow families: a high-throughput single-agent batch workflow and a staged multi-agent review workflow. The goal of the study is not to introduce a new prompting method or model architecture. Instead, we evaluate the engineering controls needed to parallelize record-level agent execution while keeping it observable, recoverable, and tunable at scale.

Across hundreds of records and thousands of model and retrieval calls, reliability depended less on prompt design alone and more on production controls: separating local worker parallelism from hosted API concurrency, enforcing per-record wall-clock budgets, distinguishing transient from non-transient failures, preserving progress through checkpoints, and recording structured operational metadata. In the single-agent workflow, hosted API concurrency was the dominant throughput bottleneck; tuning it improved throughput by nearly 5x relative to the initial baseline while preserving completion yield.

In the multi-agent workflow, a verifier stage and multi-source evidence gathering produced richer metadata but also introduced orchestration cost and partial-completion behavior. Later probes showed that record-level concurrency, verifier concurrency caps, token pressure, and reasoning effort moved the system across different throughput, cost, yield, and strict label-agreement operating points. We argue that production agent deployment should be formulated as a constrained multi-objective systems problem, rather than as model-quality maximization alone.

Keywords llm agents, agentic ai, production ai, reliability engineering, distributed systems, observability, batch processing, scientific computing

Published Jul 13, 2026

Correspondence to
Sruthi Pisipati
sruthi.pisipati@gmail.com

Open Access

Copyright © 2026 Pisipati & Javed. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Agentic AI systems are increasingly used to automate scientific, industrial, and regulated workflows. During early development, these systems are commonly evaluated through interactive applications, small test sets, or one-record-at-a-time demonstrations. Under those conditions, failures are visible and recoverable: a developer can inspect a prompt, rerun an input, adjust the output schema, or manually retry an API call.

Production environments expose a different class of problems. When the same agent is run across hundreds or thousands of records, it must coordinate concurrent workers, hosted LLM APIs, managed retrieval services such as search or vector-index systems that fetch supporting evidence for the model, cloud object storage for input files, outputs, and checkpoints, long-running execution, timeouts, retries, and partial outputs. Rare failures become routine at scale. Worse, some failures are not hard crashes. An agent stage may catch an

exception, return a fallback answer, or complete the batch while internal errors remain visible only in logs.

This paper studies those reliability problems through a real production case study. We evaluate two related workflows: a single-agent batch classification pipeline that processes each record independently, and a multi-agent review workflow that retrieves supporting evidence, synthesizes a structured prediction, verifies the prediction, attempts a bounded repair when needed, and routes outputs by confidence. We frame these workflows as production systems rather than isolated model calls. That framing follows a broader lesson from production machine learning: systems fail not only because of model behavior, but because data dependencies, validation, serving, monitoring, and operational controls are handled ad hoc [1], [2].

The underlying task is a structured multi-label classification problem over access-controlled records. Each input record contains free-text narrative fields and structured metadata. The system must return zero or more normalized labels from a controlled vocabulary, along with metadata that supports review and downstream persistence. In the multi-agent experiments, each predicted label has an identifier and a normalized three-component tuple. This paper reports only aggregate metrics and synthetic descriptions of the task structure; it does not disclose record text, prompts, retrieved evidence, raw outputs, or label content that could expose sensitive information. This abstraction is intentional: many scientific and industrial record-processing workflows have the same shape even when the label vocabulary and source records differ.

The paper makes three contributions. First, we report operational results from single-agent and multi-agent batch experiments that parallelize record processing, including throughput, yield, timeout behavior, checkpoint recovery, failure signals, token-derived cost estimates, and strict-agreement guardrails. Second, we propose a failure taxonomy and measurement set that separates final record failures from recovered internal events and transient infrastructure signals. Third, we formulate production agent tuning as a constrained multi-objective optimization problem over worker count, API concurrency, timeout budget, retry policy, checkpoint cadence, verifier stages, fan-out, and reasoning effort. This last framing is the paper's central methodological claim: production agent deployment should select a bounded operating point under reliability, cost, and auditability constraints, not simply maximize model complexity or parallelism.

The central problem is simple: an LLM agent that works once is not necessarily a production system. To deploy agents in high-volume environments, practitioners must design for the ways they fail at scale.

Table 1. *What breaks when a record-processing agent is parallelized.*

Scale-up pressure	What breaks or becomes ambiguous	Measurement or control used here
More local workers	Hosted API, retrieval, network, or storage bottlenecks dominate local parallelism	Separate worker count from API concurrency
Long-tail records	A few slow records can hold the batch open	Per-record wall-clock timeout budgets
Partial completion	A run can finish most records but still exit with failures	Completion yield, saved decisions, and failure logs
Hidden internal recovery	The final output may succeed while an agent stage recovered from an exception	Recovered-internal-error counters

Provider or policy errors	Some failures are non-transient and should not be retried indefinitely	Named error taxonomy and retry limits
Cost and token pressure	Faster configurations can increase model calls or peak token load	Token totals, peak tokens per minute (TPM), and cost per usable record
Strict agreement under concurrency	More parallel execution does not guarantee better or worse agreement	Repeat probes and strict label-set agreement

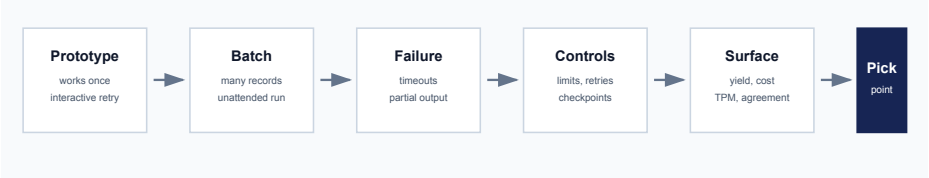


Figure 1. *Conceptual contribution of the paper. Production deployment exposes failure modes that require reliability controls and an empirical optimization surface before choosing an operating point.*

The synthesis diagram is the organizing argument for the rest of the paper: a prototype that succeeds interactively must become a measured batch system before an operating point can be selected.

2. Background

Recent work on LLM reasoning and agentic systems has shown that language models can do more than single-turn text generation. Chain-of-thought prompting demonstrated that intermediate reasoning can improve model performance on complex tasks [3]. ReAct-style agents combine reasoning with actions such as search and tool use [4]. Retrieval-augmented generation extends this pattern by grounding generation in external knowledge sources [5]. These approaches motivate agents that can retrieve candidates, reason over free text and metadata, and produce structured outputs.

Much of the agent literature, however, focuses on reasoning quality or interaction patterns. Production deployment introduces additional concerns: service quotas, network failures, retries, nondeterministic replay, malformed structured outputs, provider content filtering, and long-running partial completion. The distributed-systems literature provides a useful lens. Large-scale systems amplify rare latency and failure events; a small probability of slow or failed calls can become common when many calls are made concurrently [6]. Techniques such as bounded concurrency, timeouts, backoff with jitter, and checkpointing are standard reliability controls [7]. We apply these ideas to LLM agent workflows, where each record may trigger multiple external calls and failures may occur inside agent reasoning paths rather than at obvious service boundaries. Although the measured rates in this study are workflow-specific, the architectural pressures are common to many production agent systems: hosted model APIs, managed retrieval, batch execution, verifier stages, rate limits, and persistent outputs.

3. From Interactive Prototype To Production Batch Agent

The system began as an interactive Streamlit workflow. That interface was useful for early validation: users could load a small set of records, run the agent, inspect outputs, and manually rerun failures. This was the right level of control for proof-of-concept work, but it did not meet the production requirement: a user should be able to start a large batch, leave it

running, and return only when the system has completed or produced a clear intervention signal.

Table 2. *Prototype versus production assumptions.*

Prototype workflow	Production batch workflow
Tens of records	Hundreds or thousands of records
Human monitors execution	Unattended execution
Manual rerun on failure	Automated retry and failure classification
UI inspection during execution	Structured logs and run summaries
Session state is acceptable	Durable checkpoints and outputs
Stalls may be noticed by a user	Stalls require wall-clock detection
Success judged interactively	Success measurable and auditable

This shift changed the engineering problem. Multiprocessing introduced many simultaneous agent executions. In this paper, a worker is a local process or task that handles records, while hosted API concurrency is the explicit limit on simultaneous calls to shared external services such as hosted LLM APIs or managed retrieval. Increasing local workers improved local parallelism, but also increased pressure on hosted LLM APIs, managed search, network connections, and object storage. Production reliability therefore required separating compute parallelism from external-service concurrency.

It also changed how failures had to be represented. A production run needs structured events with record identifiers, failure types, timestamps, retry history, and final status. Internal errors recovered by the agent still need to be counted, because a clean final success count can hide important operational behavior.

4. Workflow Overview

We evaluated two related workflow families in an evolutionary line for scaling record-level agent work. The single-agent batch workflow processes each record independently using one LLM-based agent. The runner loads records from spreadsheet or cloud object storage, creates record-level tasks, executes them in parallel, and writes JSON outputs, spreadsheet outputs, failure logs, checkpoints, and run summaries.

The multi-agent workflow extends the same scaling problem with additional stages rather than serving as an unrelated alternative. It separates responsibilities across retrieval, reasoning, synthesis, verification, repair, and confidence routing. Retrieval gathers candidate evidence, synthesis assembles a final structured prediction, verification checks the proposed prediction, repair is one bounded correction attempt after a verifier objection, and confidence routing assigns the result to a review tier or escalation path. A verifier is a stage that reviews proposed classifications and either passes them, requests repair, or escalates the record for additional review. Fan-out means branching a record through multiple evidence paths or agent calls, such as free-text narrative and structured fields already present in the input record, before combining the results into one final classification. The later verifier-cap and token-pressure probes used a revised implementation of this multi-agent workflow with an explicit shared verifier-call limiter and additional token instrumentation. We treat it as an evolutionary runtime revision, not as a directly comparable third task architecture.

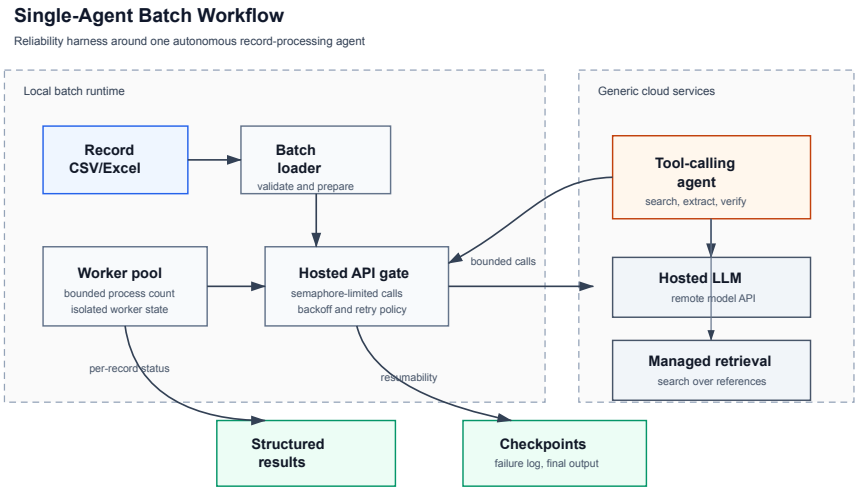


Figure 2. *Runtime reliability pattern for the single-agent batch workflow. The batch harness separates worker parallelism from hosted API concurrency and records structured outputs, failure logs, and resumable checkpoints.*

The important control in the single-agent workflow figure is the API gate between local workers and shared hosted services. It prevents the number of local record workers from becoming the same as the number of simultaneous external calls.

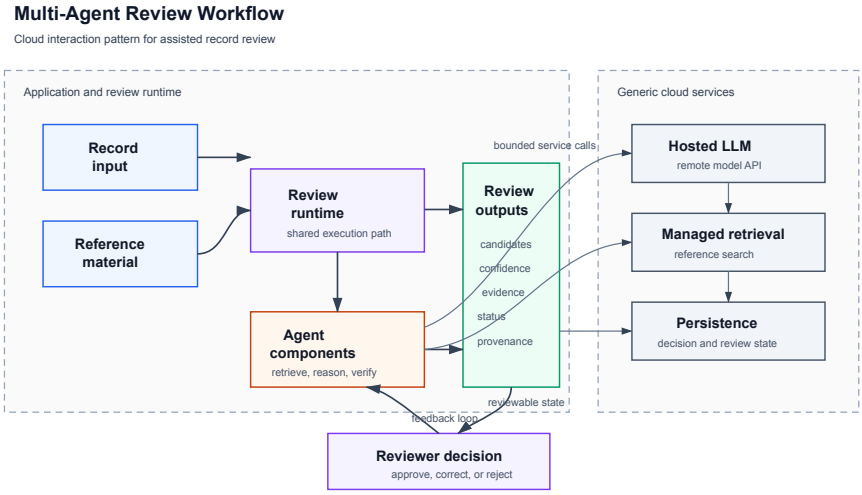


Figure 3. *Runtime reliability pattern for the multi-agent workflow. Multiple agent stages interact with hosted retrieval, LLM, and persistence services to produce confidence-aware structured outputs.*

The multi-agent workflow figure shows how review metadata is added, but it also shows why there are more shared dependencies and more places where partial completion can occur.

5. Failure Taxonomy

At production scale, “failed” is not a single state. The system needs a taxonomy that distinguishes transient infrastructure errors, provider or policy failures, timeouts, malformed outputs, recovered internal exceptions, and final unrecoverable record failures.

We organized the taxonomy around standard batch-system failure classes, such as success, retryable infrastructure errors, timeouts, and final failures. During operational debugging, we refined it to capture agent-specific signals that otherwise disappeared inside logs, such as provider policy failures, malformed model outputs, and recovered internal exceptions. The result is a reusable observability schema rather than a post-hoc list of one-off incidents.

Table 3. *Failure taxonomy used for production agent observability.*

Failure class	Description	Typical handling
`SUCCESS`	Record completed with usable structured output	Count as completed
`TRANSIENT_ERROR`	Temporary service or network issue	Retry with backoff and jitter
`RATE_LIMIT`	External quota or throttling event	Backoff and reduce concurrency
`TIMEOUT`	Record exceeded wall-clock task budget	Stop task and log structured failure
`NON_TRANSIENT_PROVIDER_ERROR`	Provider rejected request or output	Avoid unbounded retry
`OUTPUT_LIMIT_ERROR`	Model hit output or token limit	Retry with adjusted limits or classify
`VALIDATION_ERROR`	Structured response invalid or incomplete	Attempt repair or fallback
`RECOVERED_INTERNAL_ERROR`	Agent stage failed but workflow recovered	Count separately from final failures
`AGENT_INTERNAL_ERROR`	Agent failed and record could not complete	Mark final record failure
`PARTIAL_BATCH_COMPLETION`	Some records completed and some failed	Preserve successes and support targeted retry
`INTERRUPTED_RUN`	Process stopped before completion	Resume from checkpoint

This taxonomy separates final outcomes from internal operational signals. That distinction matters because an internal content-filter event may be recovered, while a batch with a perfect final success count may still contain reliability signals worth monitoring.

6. Experimental Design

The experiments were designed to evaluate operational reliability rather than broad model accuracy. We measured whether workflows completed long-running batches, preserved partial progress, exposed internal error signals, and supported configuration tuning. The experiments are best read as calibration probes for a single systems question: which bounded configuration satisfies yield, throughput, cost, token-pressure, and auditability constraints for a production agent workflow?

Table 4. *Experimental workloads.*

Workflow	Dataset	Experiment size	Purpose
Single-agent optimization sweeps	1,000-record access-controlled source file	200 records per run	Worker/API throughput and cost
Single-agent output-stability guardrail	Labeled calibration subset	12 records, 3 repeats per configuration	Strict-agreement stability under higher concurrency

Single-agent timeout tests	Access-controlled source-file subset	50 records per run	Timeout budget calibration
Single-agent checkpoint/replay tests	Access-controlled source-file subsets	300-record checkpoint run and 367-record replay audit	Checkpoint recovery and replay behavior
Multi-agent review	Access-controlled review sample	Four 50-record variant runs	Verifier and fan-out reliability
Multi-agent structured prediction	200 reference-labeled records	Two 200-record baseline runs	Multi-label classification baseline
Multi-agent concurrency stability	100 known-solvable records	3 concurrency settings x 100 records x 3 repeats = 900 record-runs	Concurrency stability and repeat consistency
Multi-agent verifier concurrency cap sweep	100 known-solvable records	5 cap settings x 100 records x 3 repeats = 1,500 record-runs	Verifier concurrency cap behavior
Multi-agent token-pressure stress tests	100 known-solvable records	4 configurations x 100 records = 400 record-runs	Reasoning-effort and token-pressure behavior

A record-run is one record processed once under one configuration and repeat. A known-solvable cohort is a deliberately selected regression set: records whose label-identifier sets matched the reference sets in a previous run and that are therefore useful for testing whether a new configuration destabilizes previously successful behavior. Selection by identifier-set agreement does not imply that the stricter three-component tuples also matched. A verifier concurrency cap is the maximum number of verifier-stage model calls allowed to run at the same time.

Comparisons within a sweep used the same selected record cohort. Most single-agent worker and API sweep points, and each token-pressure stress configuration, were observed once; experiments with repeated runs are identified explicitly. The single-agent and multi-agent campaigns used different source cohorts, execution hosts, code paths, and service-call graphs, so their absolute throughput and agreement values should not be interpreted as a head-to-head architecture benchmark. The later verifier-cap and token-pressure results should likewise be compared within their revised multi-agent runtime rather than against absolute agreement levels from the earlier multi-agent implementation.

We use *reference agreement*, *stability*, and *regression* for different questions. Reference agreement compares predictions with labels on the 200-record reference-labeled sample; it is not a comprehensive estimate of downstream accuracy. Stability asks whether repeated runs on the same records produce the same label sets. Regression probes ask whether configurations preserve behavior on records selected from prior successes. The 12-record guardrail and 100-record known-solvable cohorts therefore do not estimate population accuracy.

The repeated configurations are reported as pooled record-run counts, mean runtime, and repeat consistency. Reprocessing the same fixed cohort does not create independent samples from a broader population, so these repeats are not used to claim statistical equivalence or a population-level confidence interval. Configurations observed once provide no run-to-run variance estimate; small differences between operating points should therefore be treated as descriptive.

We report aggregate operational measurements only: configuration settings, record counts, completion status, wall-clock runtime, timeout and error counts, token totals, estimated

cost, checkpoint recovery behavior, and verifier or routing summaries. Source narratives, prompts, retrieved evidence, raw logs, per-record outputs, and checkpoint files are not published because they may contain sensitive regulated information.

Token pressure means the rate at which prompt, completion, reasoning, and embedding tokens are consumed by hosted model services. We summarize this as peak tokens per minute (TPM) over a 60-second window. Reasoning effort is a model configuration that changes the internal reasoning budget available to the model when the provider exposes such a setting. It is distinct from workflow confidence tiers, severity labels, or human review categories.

Cost estimates are market approximations based on a June 12, 2026 snapshot of public list prices: GPT-5 input tokens at USD 1.25 per 1M tokens, GPT-5 output tokens at USD 10.00 per 1M tokens, and text-embedding-3-small at USD 0.02 per 1M tokens [8], [9]. These estimates do not reflect negotiated enterprise pricing, Azure-specific billing, cached-input discounts, reserved capacity, or confidential commercial terms.

Three related controls were held outside the experimental sweeps. The API concurrency gate bounded simultaneous in-flight calls, while the runner separately classified rate-limit responses; the reported single-agent sweeps recorded no rate-limit errors, so they do not characterize behavior beyond quota saturation. Temperature remained at each workflow's configured provider-compatible setting and was not varied. Prompt caching was not instrumented as a separate treatment or cost category, and the cost estimates do not assume cached-input discounts.

In the multi-agent structured-prediction experiments, we evaluate two representations of each predicted label: its identifier and its normalized three-component tuple. We refer to strict set comparison of those representations as exact label-ID match and exact label-triplet match, respectively.

Table 5. *Metrics used in the experiments.*

Metric	Meaning
Completion yield	Successful usable records divided by attempted records
Usable throughput	Usable records divided by wall-clock runtime
Usable predictions per minute	Records with non-empty predicted label sets divided by wall-clock runtime in minutes
Timeout errors	Records terminated by wall-clock budget
Checkpoint recovery	Records preserved rather than reprocessed after interruption
Token usage	Prompt, completion, total, and embedding tokens
Estimated cost	Token-derived public-pricing approximation
Prediction yield	Records with a non-empty predicted label set divided by attempted records
Exact label-ID match	Predicted label-identifier set exactly equals the reference identifier set
Exact label-triplet match	Predicted normalized three-component tuple set exactly equals the reference tuple set
Repeat consistency	Records whose predicted label set remains stable across repeated runs
Peak TPM	Maximum observed tokens per minute in a 60-second window
Reasoning tokens	Completion-side internal reasoning tokens reported by the model provider when available

The exact-match metrics are intentionally strict. For example, if the reference label-ID set is `{A, B}`, then `{B, A}` is an exact label-ID match because the set is identical, but `{A}` fails because it is missing `B`, and `{A, B, C}` fails because it adds `C`. Separately, if a reference label has the normalized tuple `(term_1, context_1, outcome_1)`, then `(term_1, context_2, outcome_1)` is not an exact label-triplet match even when its associated label identifier is correct. This strictness makes the metrics useful for stability and regression testing, but it can understate partial or semantically close agreement.

7. Single-Agent Results

The initial 200-record single-agent baseline used 8 workers and hosted API concurrency 3. It completed 199 of 200 records, with one timeout, in 93.7 minutes. This produced 2.125 usable records per minute. Later sweeps used the same 200-record scale to estimate how much performance could be recovered through configuration tuning.

7.1. Worker Count

API concurrency was fixed at 3 while worker count varied.

Table 6. *Worker sweep with hosted API concurrency fixed at 3.*

Workers	API concurrency	Records	Successful	Runtime min	Usable records/min
3	3	200	200	63.4	3.157
4	3	200	200	62.7	3.191
6	3	200	200	63.1	3.170
8	3	200	200	61.6	3.248
12	3	200	200	59.3	3.373

Increasing worker count helped only modestly once hosted API concurrency was fixed. The change from 3 to 12 workers improved throughput from 3.157 to 3.373 usable records per minute, indicating that local worker parallelism was not the dominant bottleneck.

7.2. Hosted API Concurrency

Worker count was fixed at 12 while hosted API concurrency varied.

Table 7. *Hosted API concurrency sweep with worker count fixed at 12.*

Workers	API concurrency	Records	Successful	Runtime min	Usable records/min
12	1	200	200	168.3	1.19
12	2	200	200	81.9	2.44
12	3	200	200	54.0	3.71
12	4	200	200	51.8	3.86
12	6	200	200	38.2	5.23
12	8	200	200	27.5	7.29
12	12	200	200	19.0	10.52

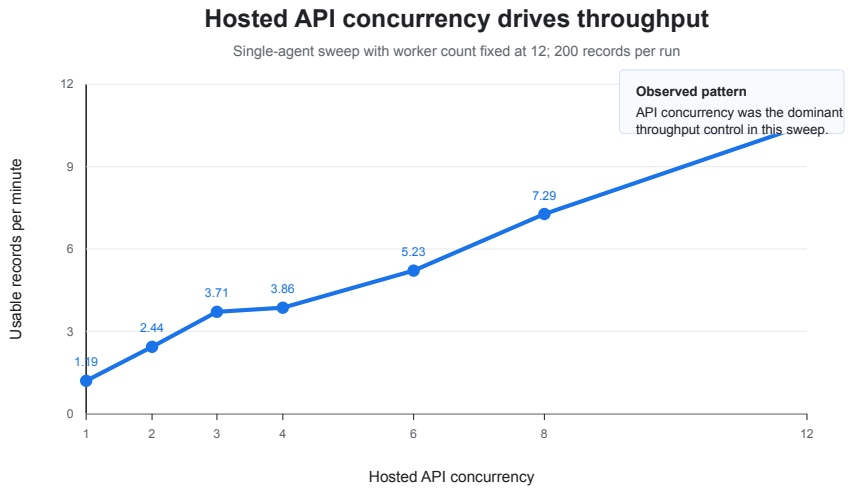


Figure 4. *Hosted API concurrency response for the single-agent workflow. With worker count fixed at 12, throughput increased from 1.19 usable records per minute at API concurrency 1 to 10.52 usable records per minute at API concurrency 12.*

Hosted API concurrency was the dominant throughput bottleneck. Throughput increased from 1.19 usable records per minute at API concurrency 1 to 10.52 at API concurrency 12, while completion yield remained 100% and no timeout, rate-limit, or connection failures were observed in these runs. Relative to the initial 8-worker/API-concurrency-3 baseline, the tuned 12-worker/API-concurrency-12 setting improved throughput by approximately 4.95x.

The best single-agent run consumed 13,745,153 chat prompt tokens, 685,051 chat completion tokens, and 6,843 embedding tokens. Using the public-pricing approximation described above, the estimated cost was approximately USD 24 for the 200-record run, or about USD 0.12 per record.

7.3. Output-Stability Guardrail, Timeouts, Checkpointing, And Replay

Because full reference labels were not available for all 1,000 records, we used a 12-record labeled calibration subset to test whether higher concurrency changed strict label agreement on a small known-labeled sample. Each configuration was run three times, producing 36 record-level comparisons per configuration. This is an output-stability guardrail, not evidence of general quality preservation.

Table 8. *Single-agent output-stability guardrail on 12 labeled records.*

Config	Workers	API concurrency	Repeats	Record-runs	Exact matches	Exact match rate	Consistency rate
`w12_a3`	12	3	3	36	33	91.67%	100.00%
`w12_a12`	12	12	3	36	34	94.44%	91.67%

The observed exact-match rate was similar across the two settings: 91.67% for `w12_a3` and 94.44% for `w12_a12`. Repeat consistency decreased from 100.00% to 91.67%, corresponding to one of 12 records. Because the subset contains only 12 labeled records, these mixed observations should be interpreted as a guardrail sanity check for concurrency changes, not as evidence of general quality preservation or as a definitive accuracy benchmark.

Timeout experiments showed why wall-clock budgets must be calibrated rather than minimized. A 60-second budget completed only 17 of 50 records and produced 33 timeout errors. A 120-second budget completed all 50 records in 23.3 minutes. A 300-second budget also completed all 50 records in 20.5 minutes. The 60-second budget was faster but unacceptable under a near-100% completion requirement.

Checkpointing avoided repeated work. In a 300-record run, a checkpoint preserved the first 200 completed records. Resuming from that checkpoint processed only the remaining 100 records, which completed successfully in 26.2 minutes. Historical replay also showed why recovered internal errors must be tracked separately from final failures. Replaying 367 records associated with historical agent-internal failures produced 367 final successes and zero top-level internal failures, but surfaced one recovered content-filter event.

8. Multi-Agent Results

The multi-agent results are organized in three phases. First, we compare review-workflow variants to understand how verification and fan-out affect reliability signals. Second, we evaluate the workflow as a multi-label classification task against reference labels. Third, we use known-solvable records to stress concurrency, verifier concurrency caps, token pressure, and reasoning effort as operational controls. The verifier-cap and token-pressure experiments in the third phase used the revised multi-agent runtime described above. This sequence separates operational reliability, strict label agreement, and regression-style pressure testing. The agreement-related rows should therefore be read according to their workload and runtime version: the 200-record baseline estimates agreement with available reference labels, while the known-solvable probes test whether previously successful cases remain stable under pressure.

8.1. Verifier And Fan-Out Runs

This table compares review-workflow variants, not reasoning-effort settings. The confidence columns report workflow-assigned confidence tiers for the proposed classification; they are not model reasoning levels or severity labels. In the table, “structured input fields” means metadata fields already present in the input record. The workflow does not require these fields; the ON/OFF runs are an ablation that tests whether adding this optional evidence path changes throughput, completion, or review metadata. These fields are not the reference labels used for scoring. The verifier columns count how many records passed verification, required one repair attempt, or were escalated.

Table 9. *Multi-agent verifier and fan-out reliability runs.*

Run	Saved/ total	Wall sec	Records/ min	Confidence high	Confidence medium	Confidence low	Verifier pass	Repair	Escalate
Verifier ON, retry 1	48/50	1188.4	2.53	22	23	3	45	3	0
Verifier OFF	49/50	978.6	3.07	31	18	0	0	0	0
Fan-out, structured fields ON	48/50	1347.2	2.23	23	19	6	44	2	2

Fan-out, structured fields OFF	48/50	1183.4	2.54	26	20	2	46	1	1

Verifier and fan-out settings produced richer review metadata, but increased orchestration cost and still produced partial completion. In the review-workflow comparison table, turning the verifier off improved throughput from 2.53 to 3.07 records per minute relative to the verifier-on run, but removed verifier pass, repair, and escalation metadata.

Table 10. *Multi-agent operational error signals.*

Run	Content-filter signals	Read-timeout signals	Connection-pool warnings	Final errors
Fan-out, structured fields OFF	6	0	1	2
Fan-out, structured fields ON	3	2	4	2
Verifier OFF	8	0	5	1
Verifier ON, retry 1	8	0	4	2

Provider content-filter signals were the most frequent named error category in the operational-error table. One fan-out configuration also produced read-timeout signals. Connection-pool warnings were operational signals but did not by themselves terminate the batch.

8.2. Multi-Label Classification Baseline

We next evaluated the multi-agent workflow as a multi-label classification task. Each input record had a reference set of target labels, and the system predicted a set of labels. Each label is represented by an identifier and a normalized three-component tuple. A prediction counted as an exact label-ID match when the predicted identifier set equaled the reference identifier set. The stricter exact label-triplet metric required the full predicted tuple set to equal the reference tuple set. Prediction yield measured whether the workflow produced at least one predicted label for a record.

These metrics evaluate agreement with the experiment's reference labels. We use exact set agreement intentionally as a strict stability measure; partial semantic correctness is outside the scope of this paper. The metrics should not be interpreted as a comprehensive assessment of downstream workflow quality, domain-level correctness, or real-world review outcomes.

Both baseline runs used medium reasoning, the verifier, and the optional structured-input evidence path; record-level concurrency was the varied setting.

Table 11. *Multi-agent 200-record multi-label classification baseline.*

Concurrency	Saved/ total	Hard errors	Prediction yield	Exact label-ID	Exact label-triplet	Wall mi	Usable predictions/ min	Est. cost
2	193/200	7	187/200 (93.5%)	96/200 (48.0%)	47/200 (23.5%)	87.3	2.14	USD 10.22

6	196/200	4	193/200 (96.5%)	100/200 (50.0%)	51/200 (25.5%)	28.8	6.70	USD 10.32

Increasing record-level concurrency from 2 to 6 improved throughput by approximately 3.1x, from 2.14 to 6.70 usable predictions per minute, and improved prediction yield from 93.5% to 96.5%. Exact label-ID agreement increased from 48.0% to 50.0%, while exact label-triplet agreement increased from 23.5% to 25.5%. Both process runs returned nonzero exit codes because a small number of records encountered provider content-filter failures, but the runs still saved most record-level decisions and produced usable aggregate results.

8.3. Known-Solvable Concurrency Stability Probe

To test whether higher record-level concurrency degraded the workflow on records it had already shown it could solve, we constructed a 100-record known-solvable cohort from the 200-record concurrency-6 baseline. The cohort included records where the predicted label-ID set exactly matched the reference label-ID set in that run. This cohort is useful for stability testing, but it is not an unbiased estimate of overall classification accuracy because it was selected from prior successes. Its purpose is closer to a regression test: a configuration that destabilizes these cases is operationally concerning, but a high score on this cohort does not imply high accuracy on arbitrary future records.

Each concurrency setting was run three times, producing 300 record-runs per setting.

Table 12. *Known-solvable concurrency stability probe.*

Concurrency	Repeats	Record-runs	Prediction yield	Exact label-ID	Exact label-triplet	Triplet consistency	Avg wall min/100	Usable predictions/min	Est. cost/100 usable
2	3	300	288/300 (96.0%)	279/300 (93.0%)	102/300 (34.0%)	93.0%	46.9	2.05	USD 4.99
6	3	300	292/300 (97.3%)	278/300 (92.7%)	96/300 (32.0%)	93.0%	16.0	6.09	USD 4.98
12	3	300	288/300 (96.0%)	281/300 (93.7%)	103/300 (34.3%)	96.0%	10.8	8.92	USD 5.01

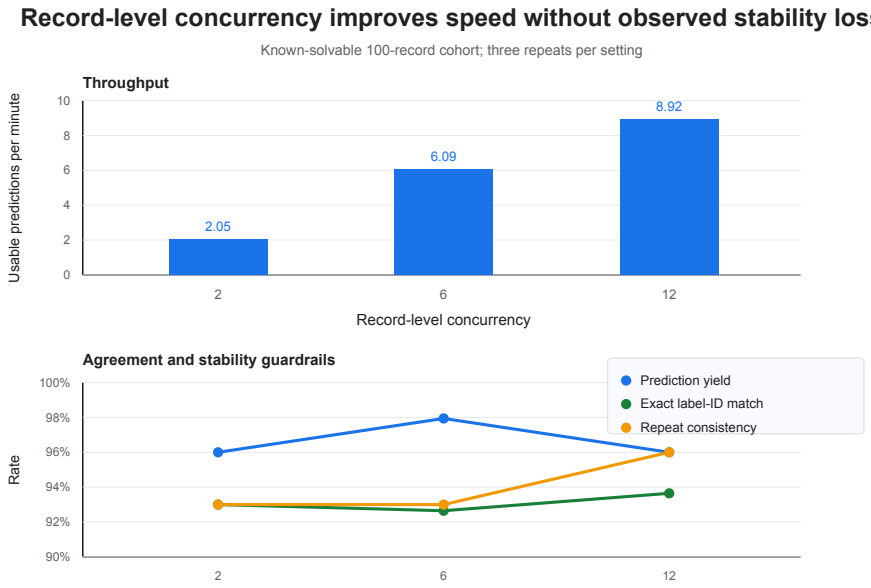


Figure 5. *Known-solvable multi-agent concurrency stability probe. Record-level concurrency improved usable predictions per minute from 2.05 to 8.92, while prediction yield, exact label-ID agreement, and repeat consistency changed only modestly across the tested settings.*

On the known-solvable cohort, increasing record-level concurrency from 2 to 12 improved throughput by approximately 4.4x, from 2.05 to 8.92 usable predictions per minute. At those endpoints, prediction yield was 96.0% in both settings, exact label-ID agreement was 93.0% and 93.7%, exact label-triplet agreement was 34.0% and 34.3%, and repeat consistency was 93.0% and 96.0%. The intermediate concurrency-6 setting had lower exact label-triplet agreement at 32.0%, so these results show no monotonic degradation across the tested settings rather than statistical equivalence. This supports workflow-level parallelism across independent records, while each record retains its own reasoning and synthesis path. It should not be read as a general claim that arbitrary parallel reasoning is always safe.

8.4. Verifier Concurrency And Token-Pressure Probes

We next asked whether strict agreement would degrade when the multi-agent workflow placed more pressure on shared hosted-model capacity. Token pressure means the volume of prompt, completion, reasoning, and embedding tokens sent through the hosted model stack per minute. We tested this pressure in two ways. First, we held batch concurrency at 10 records and varied the verifier concurrency cap, which limits how many verifier calls may run at the same time. Second, we ran a stress matrix that changed batch concurrency, verifier concurrency, reasoning effort, and optional classifier stages. These probes use the same 100-record known-solvable cohort as the preceding stability probe, so they should be interpreted as regression and pressure tests rather than unbiased accuracy estimates.

The verifier concurrency cap sweep used medium reasoning effort, 100 records, and three repeats per setting. A lower cap serialized verifier calls more aggressively; a higher cap allowed verifier calls to fan out more freely. Because this sweep uses the known-solvable cohort, the exact label-triplet values in the verifier-cap table are pressure-test and repeat-stability measurements, not an unbiased accuracy estimate.

Table 13. *Verifier concurrency cap sweep.*

Verifier concurrency cap	Record-runs	Prediction yield	Exact label-triplet	Repeat consistency	Avg records/min	Peak total TPM	Est. cost/100 usable
1	300	86.3%	80.7%	81.0%	9.69	329k	USD 5.63
3	300	89.0%	84.0%	91.0%	10.30	393k	USD 5.63
6	300	86.0%	80.3%	83.0%	10.34	412k	USD 5.61
10	300	88.0%	84.3%	86.0%	10.51	428k	USD 5.59
20	300	88.0%	83.3%	82.0%	9.91	389k	USD 5.61

Increasing verifier concurrency did not show a monotonic relationship with strict agreement. The verifier concurrency cap of 3 had the strongest repeat consistency, while the verifier concurrency cap of 10 had the highest exact label-triplet agreement and throughput. The operational lesson is that verifier calls should have an explicit concurrency cap, but the cap should be tuned empirically rather than assumed to be as high as possible.

The stress probe then compared a medium-reasoning baseline against high reasoning, higher record-level concurrency, and a maximum-pressure setting that combined high reasoning, higher batch concurrency, higher verifier concurrency, and an additional classifier stage. Like the verifier-cap sweep, the token-pressure stress table is a regression and pressure test on previously successful records.

Table 14. *Token-pressure stress probe on the known-solvable cohort.*

Config	Batch/ verifier concurrency	Reasoning/ extra stage	Prediction yield	Exact label-triplet	Records/ min	Reasoning tokens	Peak total TPM	Est. cost/100 usable
Baseline	10 / 3	Medium / off	88/100	82/100	10.18	176k	364k	USD 5.61
High reasoning confirmation	10 / 3	High / off	88/100	83/100	7.84	312k	288k	USD 7.01
High concurrency	20 / 10	Medium / off	86/100	79/100	16.69	173k	707k	USD 5.63
Maximum pressure	20 / 20	High / on	89/100	84/100	13.71	308k	576k	USD 6.95

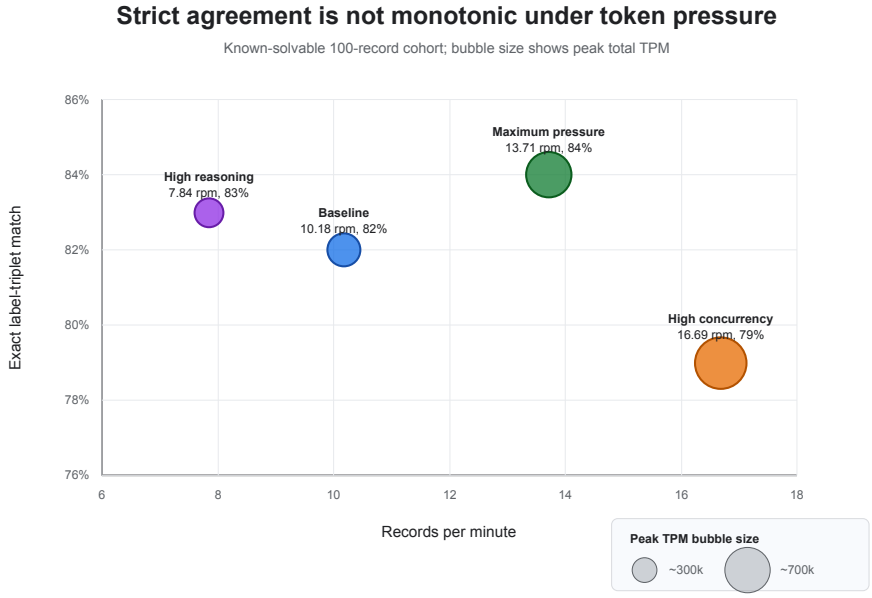


Figure 6. *Multi-agent token-pressure operating points. Higher concurrency and reasoning effort changed throughput, cost, and peak token-per-minute pressure, but exact label-triplet agreement did not move monotonically with any single knob.*

One high-reasoning run in the first stress matrix ended as a partial operational run. We excluded that partial result from the table and reran the high-reasoning configuration successfully as a confirmation run.

The stress results were more nuanced than the hypothesis that higher token pressure would necessarily reduce strict agreement. High reasoning increased reasoning-token usage by approximately 1.8x and cost by roughly 25% relative to the medium baseline, while reducing throughput from 10.18 to 7.84 records per minute; exact label-triplet agreement remained similar. Higher concurrency with medium reasoning increased throughput to 16.69 records per minute but showed a small drop in exact label-triplet agreement. The maximum-pressure configuration had the highest exact label-triplet agreement among the stress settings, but at higher cost and higher peak token pressure. These results support treating token pressure as an optimization variable, not as a one-directional failure mechanism.

We also computed an observed Pareto set from the revised-runtime multi-agent runs that used the same 100-record known-solvable cohort and reported exact label-triplet agreement. This includes the verifier concurrency cap sweep and token-pressure stress runs, but excludes the earlier multi-agent concurrency-stability phase because it used a different runtime version. The Pareto calculation compares cost per usable record, throughput, prediction yield, and strict agreement, following the standard multi-objective optimization idea of non-dominated alternatives [10]. This is not a global frontier over all possible settings or an unbiased accuracy frontier; it is the non-dominated set among the comparable calibration rows we ran.

Table 15. *Observed multi-agent Pareto-relevant operating points.*

Operating point	Role	Prediction yield	Exact label-triplet	Records/min	Peak TPM	Est. cost/100 usable
Verifier concurrency cap 3	Illustrative balanced point	89.0%	84.0%	10.30	393k	USD 5.63
Verifier concurrency cap 10	Cost/throughput edge	88.0%	84.3%	10.51	428k	USD 5.59
High concurrency	Latency-biased point	86.0%	79.0%	16.69	707k	USD 5.63
Maximum pressure	Stress-bound point	89.0%	84.0%	13.71	576k	USD 6.95

Within the observed rows, we use the verifier concurrency cap of 3 as an illustrative balanced point because it preserved the highest prediction yield and repeat consistency in the concurrency cap sweep while bounding verifier fan-out. The verifier concurrency cap of 10 was slightly cheaper and faster in aggregate, but with lower repeat consistency and higher peak token pressure. This descriptive comparison does not establish that cap 3 is statistically superior or a universal default. The high-concurrency and maximum-pressure rows remain useful frontier points for latency- or stress-oriented deployments.

Taken together, these operating points motivate the framework below: choose configurations by constraints and observed tradeoffs, rather than by maximizing any single knob.

9. Optimization Framework

The experiments suggest that production agent design should be treated as a constrained optimization problem. Organizations usually want lower cost, higher throughput, reliable completion, and auditability, but these goals can conflict. Increasing parallelism can improve runtime while increasing external-service pressure. Reducing timeout budgets can make a batch appear faster while converting slow-but-recoverable records into failures. Adding verifier or fan-out stages can improve review metadata while increasing model calls, latency, and possible failure points. This makes the framework more than a tuning checklist: each configuration maps to an observed operating point with measurable benefits and constraint violations.

The objective is not to maximize the number of agents, workers, retries, or reasoning steps. Let $x = (W, A, T, R, K, V, C_v, F, E)$ denote a configuration from the candidate set $\mathcal{X}$ for workload D. We define a feasible set using completion yield y_D, strict agreement g_D, cost

per usable record c_D, peak token pressure p_D, final failure rate f_D, and an indicator a for required auditability controls:

$$\mathcal{F}(D) = \{x \in \mathcal{X} : y_D(x) \geq Y_{\min},\, g_D(x) \geq G_{\min},\, c_D(x) \leq C_{\max},\, p_D(x) \leq P_{\max},\, f_D(x) \leq F_{\max},\, a(x) = 1\}. \tag{1}$$

The agreement constraint g_D must be estimated on an appropriate reference-labeled calibration set. Agreement measured on the selected known-solvable cohort is a regression signal and must not be substituted for a population-level quality threshold. When representative labels are unavailable, the framework cannot claim that constraint has been validated.

Among feasible configurations, the primary objective is usable throughput q_D, where u_D is the number of usable completed records and t_D is wall-clock time:

$$x^\star \in \arg\max_{x \in \mathcal{F}(D)} q_D(x), \qquad q_D(x) = \frac{u_D(x)}{t_D(x)}. \tag{2}$$

When several configurations are feasible, we compare them in the Pareto sense over $(-q_D, c_D, e_D, r_D, b_D)$, where e_D is recovered-internal-error rate, r_D is reprocessing cost, and b_D is review burden. This formulation avoids inventing universal weights: local policy sets the hard thresholds and chooses among non-dominated operating points.

Table 16. *Agent configuration variables.*

Variable	Meaning	Example values
`W`	Local worker or process count	3, 4, 6, 8, 12
`A`	Hosted LLM/API concurrency limit	1, 2, 3, 4, 6, 8, 12
`T`	Per-record wall-clock timeout budget	60s, 120s, 300s
`R`	Retry count and retry policy	none, timeout-only, transient with backoff
`K`	Checkpoint frequency	every 25, 50, or 100 completed records
`V`	Verifier stage	on, off
`C_v`	Verifier concurrency cap	1, 3, 6, 10, 20
`F`	Fan-out across evidence sources	on, off
`E`	Model or reasoning-effort setting	low, medium, high

A practical calibration loop has eight steps: select a representative workload; run a conservative baseline; sweep one control at a time; record run manifests; estimate the observed response surface; reject configurations that violate hard constraints; identify approximate Pareto-efficient configurations; and choose an operating point that matches the organization's goal. In this study, we use observed sweeps to illustrate the tradeoff structure rather than claiming a complete global Pareto frontier over all possible configurations.

This framing changes the design question from "How many workers should we run?" to "Which configuration satisfies the reliability target at the lowest acceptable time and cost?" The single-agent experiments showed that increasing `W` alone produced diminishing returns once `A` was fixed, while increasing `A` within quota produced much larger gains. The timeout experiment showed that a faster configuration can be invalid if it violates completion-yield constraints. The multi-agent experiments showed that review metadata and record-level concurrency must be evaluated jointly against throughput, yield, cost, and stability.

The verifier concurrency cap and token-pressure probes further refine this framework. Higher concurrency increased peak token-per-minute pressure, and higher reasoning effort increased reasoning-token usage and cost, but neither variable produced a simple monotonic degradation in exact label agreement. Each configuration moved the operating point across throughput, cost, token pressure, prediction yield, and strict agreement. Production

tuning should therefore be measured as a constrained optimization problem rather than decided by intuition alone.

The same framework can support an interactive decision-support widget. The widget is a supplemental artifact and a demonstration of the paper's operating-point methodology, not a live deployment interface. Rather than launching expensive LLM calls for every user interaction, the widget uses saved aggregate calibration runs to simulate expected outcomes without exposing record-level source data. Users can adjust worker count, API concurrency, verifier concurrency cap, reasoning effort, and token-pressure constraints, then inspect observed throughput, cost, yield, strict agreement, and peak token pressure. The current widget is static: it does not launch LLM calls or process new records. It lets users explore saved aggregate calibration results. A future internal version could trigger new calibration runs inside an approved environment.

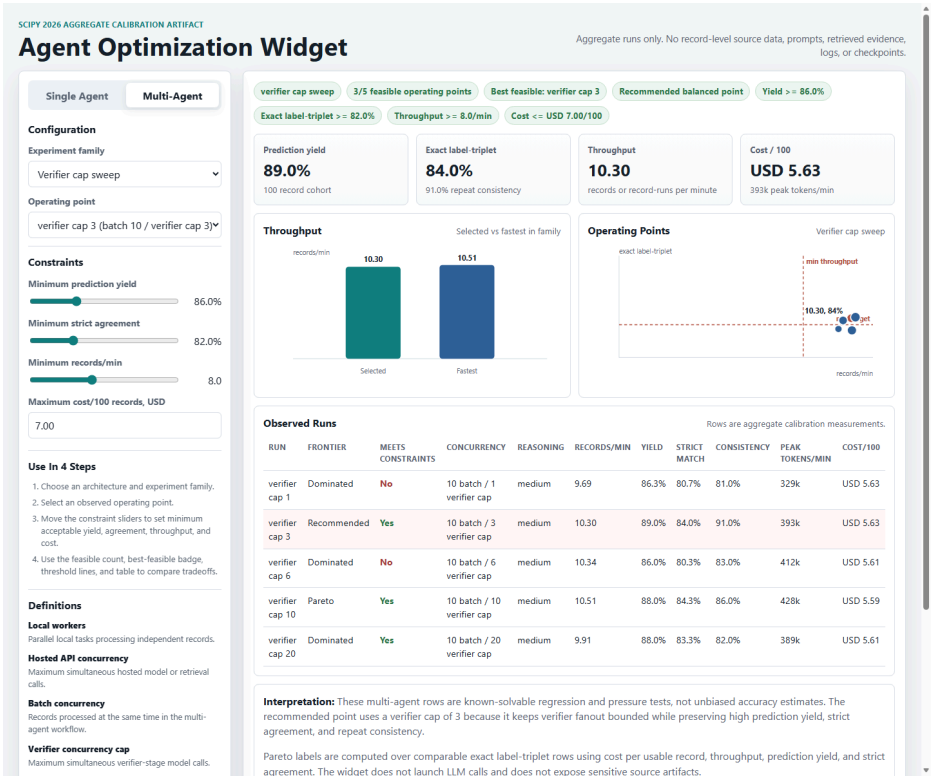


Figure 7. *Static decision-support widget for exploring aggregate calibration results. The widget is a supplemental artifact that uses saved operating points rather than live LLM calls or record-level source data.*

10. Reliability Patterns

The experiments point to several production patterns.

Standard distributed-systems controls need stage-aware semantics in an agent workflow. Retrying a deterministic service operation is intended to reproduce the same result; retrying an LLM call can produce a different candidate. A verifier then evaluates that new candidate, so replay can change both the verifier input and its verdict. Evidence fan-out multiplies stochastic calls and makes record latency depend on the slowest branch, while a failed branch may be hidden by a successful fallback or synthesis path. For this reason, the workflow must bound and log record attempts, evidence branches, and verifier calls separately, and must distinguish recovered stage failures from final record failures.

First, separate compute parallelism from API concurrency. Local worker count should not directly determine simultaneous hosted API calls. A bounded semaphore or equivalent limiter should protect each shared dependency, including hosted LLM APIs, managed search, object storage, and secret stores.

Second, classify transient and non-transient failures. Connection resets and read timeouts may succeed on retry. Provider content-filter rejections, invalid prompts, and input-specific policy failures usually should not be retried indefinitely. Retries should use bounded exponential backoff with jitter.

Third, use wall-clock task budgets. Individual records can run much longer than the median, and a small number of slow tasks can hold a batch open indefinitely. Timeouts make those stalls observable, but the budget must be calibrated against completion-yield requirements.

Fourth, checkpoint long-running batches. Checkpoints preserve progress across interruption, reduce reprocessing cost, and reduce exposure to nondeterministic reruns.

Fifth, distinguish recovered internal errors from final failures. Agent stages may recover through fallback paths. Those events should not necessarily mark the record as failed, but they should be counted separately so clean final success counts do not hide reliability signals.

Finally, design for partial completion. A run that completes 196 of 200 records is operationally different from a run that crashes before producing outputs. The system should preserve successful records, classify failed records, and provide a path for targeted retry or review.

11. Data And Artifact Availability

The experiments were conducted in an internal secured environment using access-controlled records and production-like cloud dependencies. Record-level source artifacts, raw narratives, prompts, retrieved evidence, raw logs, per-record outputs, and checkpoints are not public because they may contain sensitive regulated information. Shareable paper-supporting artifacts include the proceedings source, embedded figures, aggregate result tables, and a static decision-support widget built from aggregate calibration rows. Experiment scripts and broader run manifests can be shared only through approved internal access controls.

The reproducibility claim is therefore procedural rather than direct. The public artifact supports inspection of the measurement framework, operating-point tables, figures, and static widget behavior. It does not support direct reproduction of the exact numerical results from the original records, prompts, raw logs, outputs, or checkpoints. A reader with an equivalent authorized dataset and comparable hosted services could reproduce the procedure: define a workload, run baseline and sweep configurations, record the same aggregate measurements, and choose a bounded operating point under local constraints.

12. Limitations

This study has several limitations:

- **Scope.** The experiments were conducted in one regulated workflow. Exact failure rates may differ in other scientific or industrial settings.
- **Provider drift.** The systems used hosted LLM APIs and managed cloud services whose behavior can change over time as providers update models, infrastructure, and content policies.

- **Replay nondeterminism.** Replaying historical agent-internal failures did not reproduce the same final failures. This is an important reliability result, but it also means some historical failure modes could not be deterministically recreated under current conditions.
- **Disclosure limits.** Some sensitive examples cannot be disclosed.
- **Uncertainty treatment.** Repeated configurations report pooled counts, mean runtime, and consistency, but not run-level standard deviations or confidence intervals. Repeats on the same fixed cohort are clustered observations rather than independent population samples, and configurations observed once provide no variance estimate. Small differences between nearby operating points should be interpreted cautiously; a confirmatory study should pre-specify repeated runs and an uncertainty model.
- **Limited labeled calibration.** The single-agent labeled calibration used only 12 records and should not be interpreted as a definitive accuracy benchmark.
- **Reference-label interpretation.** The 200-record multi-agent structured-prediction probe evaluates agreement with internal reference labels, not broader downstream correctness. The exact-match metrics are intentionally strict: a record counts as correct only when the full predicted label set matches the reference label set. They do not give partial credit for predictions that are close or partly correct.
- **Known-solvable selection.** The 100-record concurrency stability, verifier concurrency cap, and token-pressure probes were selected from prior successes. They are regression and pressure tests, not unbiased accuracy samples. The verifier concurrency cap sweep used three repeats, while the token-pressure stress matrix used one completed run per stress configuration. One high-reasoning stress attempt ended as a partial operational run and was replaced with a successful confirmation run in the reported table.

13. Conclusion

LLM agents that work in interactive prototypes are not automatically production-ready. When deployed in high-volume batch workflows, they encounter the same problems as other distributed systems: concurrency bottlenecks, external service limits, transient failures, non-transient provider errors, stalled tasks, partial completion, and hidden internal exceptions.

Through single-agent and multi-agent structured classification workflows, we showed that reliability requires explicit engineering controls around the agent. Separating compute parallelism from API concurrency improved scalability. Wall-clock budgets made stalls observable. Checkpointing preserved progress. Failure taxonomies and structured metadata made it possible to distinguish final record failures from recovered internal errors.

The final multi-agent probes showed that record-level concurrency, verifier concurrency caps, reasoning effort, and optional classification stages can be tuned as operational controls. On known-solvable cohorts, these controls changed throughput, cost, peak token pressure, and strict label agreement, but strict agreement did not move monotonically with any single setting. The useful production question is therefore not which knob should be maximized, but which bounded configuration satisfies the required yield and agreement targets at acceptable cost.

The broader lesson is that production agentic AI is not only a modeling problem. It is a systems engineering problem and an optimization problem. Robust deployment requires designing for failure from the beginning so agent workflows are observable, recoverable, tunable, and auditable at scale.

References

[1] D. Sculley *et al.*, "Hidden Technical Debt in Machine Learning Systems," in *Advances in Neural Information Processing Systems*, 2015.

[2] D. Baylor *et al.*, "TFX: A TensorFlow-Based Production-Scale Machine Learning Platform," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017, pp. 1387–1395. doi: 10.1145/3097983.3098021.

[3] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *arXiv preprint arXiv:2201.11903*, 2022, doi: 10.52202/068431-1800.

[4] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2210.03629.

[5] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, 2020. doi: 10.48550/arXiv.2005.11401.

[6] J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013, doi: 10.1145/2408776.2408794.

[7] M. Brooker, "Exponential Backoff and Jitter." [Online]. Available: https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/

[8] OpenAI, "Introducing GPT-5 for Developers." [Online]. Available: https://openai.com/index/introducing-gpt-5-for-developers/

[9] OpenAI, "New Embedding Models and API Updates." [Online]. Available: https://openai.com/index/new-embedding-models-and-api-updates/

[10] K. Miettinen, *Nonlinear Multiobjective Optimization*. Kluwer Academic Publishers, 1999. doi: 10.1007/978-1-4615-5563-6.